

Section E Tasks

Lab Task 1: Library Management System

Scenario:

A digital library stores books by Accession Number and needs quick lookup, availability checks, and bulk operations for lending and audits.

Task Requirements:

Each node should contain:

- AccessionNumber (int, unique key)
- BookTitle (string)
- Author (string)
- Availability (bool)

Core Functionalities:

- addBook(int acc, string title, string author, bool available)
- searchBook(int acc)
- removeBook(int acc)
- displayBooksInOrder()
- countAvailableBooks()

Advanced Operations:

Advanced Operations:

- findNextBook(int accessionNumber) // next higher accession
- getBooksByAuthor(string author) -> vector<Book>
- getHeight()

Implementation Guidelines:

- Implement BookNode and LibraryBST classes.
- Use recursion; do not allow duplicate accession numbers.
- Availability updates should be supported.

Note: Some testcases have vectors in them. You need to replace them with an appropriate list or array

Google Test Cases:

```
TEST(LibraryBSTTest, AddAndSearchBook) {
    LibraryBST bst;
    bst.addBook(101, "DSA", "Goodrich", true);
    bst.addBook(102, "AI", "Russell", false);
    Book b = bst.searchBook(101);
    EXPECT_EQ(b.BookTitle, "DSA"); // Expected correct book
    EXPECT_TRUE(b.Availability);
}

TEST(LibraryBSTTest, FindNonExistingBook) {
    LibraryBST bst;
    bst.addBook(101, "DSA", "Goodrich", true);
    Book b = bst.searchBook(999);
    EXPECT_EQ(b.BookTitle, ""); // Expected not found
}

TEST(LibraryBSTTest, RemoveBook) {
    LibraryBST bst;
    bst.addBook(1, "A", "Auth", true);
    bst.addBook(2, "B", "Auth2", true);
    bst.removeBook(1);
    EXPECT_EQ(bst.searchBook(1).BookTitle, ""); // Expected removed
}

TEST(LibraryBSTTest, CountAvailableBooks) {
    LibraryBST bst;
    bst.addBook(11, "A", "X", true);
    bst.addBook(12, "B", "Y", false);
    bst.addBook(13, "C", "Z", true);
    EXPECT_EQ(bst.countAvailableBooks(), 2); // Expected two available
}

TEST(LibraryBSTTest, FindNextBook) {
    LibraryBST bst;
    bst.addBook(10, "A", "X", true);
    bst.addBook(20, "B", "Y", true);
    bst.addBook(30, "C", "Z", true);
    Book next = bst.findNextBook(15);
    EXPECT_EQ(next.AccessionNumber, 20); // Expected next higher accession is 20
}

TEST(LibraryBSTTest, BooksByAuthor) {
```

```

    LibraryBST bst;
    bst.addBook(1, "B1", "Auth1", true);
    bst.addBook(2, "B2", "Auth1", true);
    vector<Book> v = bst.getBooksByAuthor("Auth1");
    EXPECT_EQ(v.size(), 2); // Expected two books by Auth1
}

TEST(LibraryBSTTest, InorderAndHeight) {
    LibraryBST bst;
    bst.addBook(5, "X", "A", true);
    bst.addBook(2, "Y", "B", true);
    bst.addBook(8, "Z", "C", true);
    vector<int> ino = bst.getInorderAccessionNumbers();
    EXPECT_EQ(ino, (vector<int>{2,5,8})); // Expected sorted order
    EXPECT_EQ(bst.getHeight(), 2); // Expected height
}

```

Lab Task 2: Binary Search Tree Range Queries and Statistics

You are part of a performance-analytics team maintaining an **in-memory file index** that stores file records in a **Binary Search Tree (BST)** for fast lookup by file size or ID.

Your analytics module must now support **range-based reports** and **statistical summaries**, such as:

- “Count how many files are between 100 KB and 500 KB.”
- “What is the total storage consumed by files smaller than 1 MB?”
- “Which file is the k-th smallest in size?”
- “What is the average file size in the index?”

These operations must be implemented **efficiently**, using **recursive traversal** and without converting the BST to arrays or using STL

All computations must rely solely on **tree traversal** and **basic arithmetic**.

Data Model

```
class BSTNode {
public:
    int key;           // file size or unique file ID
    std::string name;  // file name
    BSTNode* left;
    BSTNode* right;

    BSTNode(int k, std::string n)
        : key(k), name(n), left(nullptr), right(nullptr) {}
};
```

Requirements

You are required to:

1. Implement the **core BST operations**:

- `insert(int key, std::string name)`
- `inorderTraversal(BSTNode* root, std::ostream& out)`
- `getHeight(BSTNode* root)`
- `countNodes(BSTNode* root)`

2. Extend the BST to support **statistical and range-based queries**:

- Count how many nodes fall within a numeric range.
- Compute the sum of all keys within a range.

- Find the *k-th smallest* element using in-order traversal.
 - Compute basic aggregate statistics (min, max, average).
3. The implementation must:
- Use **recursion** and **integer counters/pointers** for traversal state.
 - Avoid using any STL containers or dynamic lists.
 - Return accurate results even for empty trees or out-of-range queries.

Functions to be implemented

1. Count Nodes Within a Range: Counts the number of nodes with **key** between **low** and **high** (inclusive).

```
int countInRange(BSTNode* root, int low, int high);
```

2. Sum of Keys in a Range: Computes the sum of all keys lying within the given range.

```
int sumInRange(BSTNode* root, int low, int high);
```

3. k-th Smallest Element: Finds the k-th smallest node using in-order traversal

```
BSTNode* findKthSmallest(BSTNode* root, int k, int& count);
```

4. Basic Tree stats: functions

```
int findMin(BSTNode* root);  
int findMax(BSTNode* root);  
float computeAverage(BSTNode* root);
```

5. Combined Reporting Function

```
void printRangeReport(BSTNode* root, int low, int high);
```

Example Output:

```
Range: 100 - 500 KB  
Count: 5  
Sum: 1450  
Average: 290.0
```

Test Cases

```
#include "gtest/gtest.h"
#include "BSTRangeQueries.h"
#include <sstream>
#include <cmath>

//
// --- Basic Functional Tests ---
//

TEST(BSTRangeQueriesTest, CountAndSumInRangeBasic) {
    BST tree;
    int keys[] = {40, 20, 10, 30, 60, 50, 70};
    for (int i = 0; i < 7; i++)
        tree.insert(keys[i], "f");

    EXPECT_EQ(countInRange(tree.root, 15, 55), 4); // 20,30,40,50
    EXPECT_EQ(sumInRange(tree.root, 15, 55), 140);
}

TEST(BSTRangeQueriesTest, RangeOutsideBounds) {
    BST tree;
    int keys[] = {5, 10, 15};
    for (int i = 0; i < 3; i++)
        tree.insert(keys[i], "x");

    EXPECT_EQ(countInRange(tree.root, 100, 200), 0);
    EXPECT_EQ(sumInRange(tree.root, 100, 200), 0);
}

TEST(BSTRangeQueriesTest, RangeCoveringAllKeys) {
    BST tree;
    int keys[] = {10, 20, 30, 40, 50};
    for (int i = 0; i < 5; i++)
        tree.insert(keys[i], "y");

    EXPECT_EQ(countInRange(tree.root, 0, 100), 5);
    EXPECT_EQ(sumInRange(tree.root, 0, 100), 150);
}
```

```

//
// --- K-th Smallest Tests ---
//

TEST(BSTRangeQueriesTest, KthSmallestElement) {
    BST tree;
    int keys[] = {50, 30, 70, 20, 40, 60, 80};
    for (int i = 0; i < 7; i++)
        tree.insert(keys[i], "v");

    int count = 0;
    BSTNode* third = findKthSmallest(tree.root, 3, count);
    ASSERT_NE(third, nullptr);
    EXPECT_EQ(third->key, 40);
}

TEST(BSTRangeQueriesTest, KthSmallestOutOfBounds) {
    BST tree;
    int keys[] = {10, 20, 30};
    for (int i = 0; i < 3; i++)
        tree.insert(keys[i], "n");

    int count = 0;
    BSTNode* tooLarge = findKthSmallest(tree.root, 10, count);
    EXPECT_EQ(tooLarge, nullptr);
}

TEST(BSTRangeQueriesTest, KthSmallestFirstAndLast) {
    BST tree;
    int keys[] = {5, 10, 15, 20, 25};
    for (int i = 0; i < 5; i++)
        tree.insert(keys[i], "f");

    int count = 0;
    BSTNode* first = findKthSmallest(tree.root, 1, count);
    EXPECT_EQ(first->key, 5);

    count = 0;
    BSTNode* last = findKthSmallest(tree.root, 5, count);

```

```

    EXPECT_EQ(last->key, 25);
}

//
// --- Empty and Single Node Tests ---
//

TEST(BSTRangeQueriesTest, EmptyTreeQueries) {
    BST tree;
    EXPECT_EQ(countInRange(tree.root, 0, 100), 0);
    EXPECT_EQ(sumInRange(tree.root, 0, 100), 0);
    int c = 0;
    EXPECT_EQ(findKthSmallest(tree.root, 1, c), nullptr);
}

TEST(BSTRangeQueriesTest, SingleNodeRange) {
    BST tree;
    tree.insert(25, "single");

    EXPECT_EQ(countInRange(tree.root, 25, 25), 1);
    EXPECT_EQ(sumInRange(tree.root, 25, 25), 25);
    EXPECT_EQ(countInRange(tree.root, 10, 20), 0);
}

//
// --- Min, Max, and Average Tests ---
//

TEST(BSTRangeQueriesTest, MinMaxAverage) {
    BST tree;
    int keys[] = {25, 15, 50, 10, 22, 35, 70};
    for (int i = 0; i < 7; i++)
        tree.insert(keys[i], "x");

    EXPECT_EQ(findMin(tree.root), 10);
    EXPECT_EQ(findMax(tree.root), 70);

    float avg = computeAverage(tree.root);
    float expected = (25 + 15 + 50 + 10 + 22 + 35 + 70) / 7.0;
    EXPECT_NEAR(avg, expected, 0.001);
}

```



```

}

TEST(BSTRangeQueriesTest, MinMaxSingleNode) {
    BST tree;
    tree.insert(100, "only");
    EXPECT_EQ(findMin(tree.root), 100);
    EXPECT_EQ(findMax(tree.root), 100);
}

//
// --- Boundary Conditions and Duplicates ---
//

TEST(BSTRangeQueriesTest, RangeExactlyAtBoundary) {
    BST tree;
    int keys[] = {10, 20, 30, 40, 50};
    for (int i = 0; i < 5; i++)
        tree.insert(keys[i], "t");

    EXPECT_EQ(countInRange(tree.root, 10, 30), 3); // includes 10,20,30
    EXPECT_EQ(sumInRange(tree.root, 10, 30), 60);
}

TEST(BSTRangeQueriesTest, RangeSingleBoundaryHit) {
    BST tree;
    int keys[] = {5, 15, 25, 35, 45};
    for (int i = 0; i < 5; i++)
        tree.insert(keys[i], "b");

    EXPECT_EQ(countInRange(tree.root, 45, 45), 1);
    EXPECT_EQ(sumInRange(tree.root, 45, 45), 45);
}

TEST(BSTRangeQueriesTest, DuplicateKeysHandledProperly) {
    BST tree;
    // If insert() ignores duplicates, we only get 3 nodes
    tree.insert(20, "A");
    tree.insert(20, "B"); // duplicate
    tree.insert(10, "C");
    tree.insert(30, "D");
}

```

```

    EXPECT_EQ(countNodes(tree.root), 3);
    EXPECT_EQ(countInRange(tree.root, 10, 30), 3);
}

//
// --- Structural and Large Tree Tests ---
//

TEST(BSTRangeQueriesTest, LeftSkewedTree) {
    BST tree;
    int keys[] = {50, 40, 30, 20, 10};
    for (int i = 0; i < 5; i++)
        tree.insert(keys[i], "s");

    EXPECT_EQ(countInRange(tree.root, 15, 35), 2); // 20,30
    EXPECT_EQ(sumInRange(tree.root, 15, 35), 50);
}

TEST(BSTRangeQueriesTest, RightSkewedTree) {
    BST tree;
    int keys[] = {10, 20, 30, 40, 50};
    for (int i = 0; i < 5; i++)
        tree.insert(keys[i], "r");

    EXPECT_EQ(countInRange(tree.root, 25, 45), 2); // 30,40
    EXPECT_EQ(sumInRange(tree.root, 25, 45), 70);
}

TEST(BSTRangeQueriesTest, LargeTreePerformanceCheck) {
    BST tree;
    for (int i = 1; i <= 1000; i++)
        tree.insert(i, "v");

    // Full range
    EXPECT_EQ(countInRange(tree.root, 1, 1000), 1000);

    // Partial range
    EXPECT_EQ(countInRange(tree.root, 400, 600), 201);
}

```

```

    int s = sumInRange(tree.root, 1, 10);
    EXPECT_EQ(s, 55); // 1+2+3+...+10
}

//
// --- Average and Mixed Range Tests ---
//

TEST(BSTRangeQueriesTest, ComputeAveragePartialRange) {
    BST tree;
    int keys[] = {5, 10, 15, 20, 25};
    for (int i = 0; i < 5; i++)
        tree.insert(keys[i], "v");

    int sum = sumInRange(tree.root, 10, 20);
    int count = countInRange(tree.root, 10, 20);
    float avg = (float)sum / count;

    EXPECT_EQ(sum, 45); // 10+15+20
    EXPECT_EQ(count, 3);
    EXPECT_NEAR(avg, 15.0, 0.01);
}

TEST(BSTRangeQueriesTest, PrintRangeReportVerification) {
    BST tree;
    int keys[] = {100, 200, 300, 400, 500};
    for (int i = 0; i < 5; i++)
        tree.insert(keys[i], "x");

    // Just verify no crash or bad output
    printRangeReport(tree.root, 150, 450);
}

```